



Task – Hibernate Lec 3

@GeneratedValue في ID وطرق توليد الـ @GeneratedValue

```
GenerationType.AUTO  
GenerationType.IDENTITY  
GenerationType.SEQUENCE  
GenerationType.TABLE
```

أولاً، الأربعة دول:

بيحدوا إزاي قيمة الـ **Primary Key** هتولد تلقائياً عند INSERT.

مثلاً:

```
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String name;  
}
```

```
Employee employee = new Employee();  
employee.setName("Ahmed");  
  
employeeRepository.save(employee);
```

لو عملت:

أنت لم تحدد **id**.

Hibernate + Database مسؤولين عن توليد الـ **id**.

GenerationType.AUTO

what is it?

AUTO معناها:

سيب **JPA/Hibernate** يختار أفضل **Strategy** مناسبة للـ **Database** المستخدمة.

أنت لا تحدد هل سيتم استخدام:

• Identity

• Sequence

• أو Strategy أخرى

Hibernate يقرر بناءً على الـ **Database** والـ **Dialect** والإمكانيات المتاحة.

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
```

مثال:

```
JPA
↓
AUTO
↓
حدد طريقة التوليد المناسبة Hibernate
↓
Database
```

بمعنى:

نقطة مهمة **AUTO** لا تعني بالضرورة **IDENTITY**.

الـ **Strategy** الفعلية التي يستخدمها **Hibernate** تعتمد على البيئة وإصدار

Hibernate والـ **Database**.

When to use it?

تستخدمها عندما:

- لا يهمك تحديد آلية التوليد بالتفصيل .
- تريد **Hibernate** يختار **Strategy** مناسبة .
- المشروع قد يعمل على أكثر من **Database** .
- تريد تقليل الـ Database-specific configuration .
- لكن في المشاريع التي تحتاج تحكمًا واضحًا في الأداء وسلوك الـ **ID generation**، غالبًا ستختار **Strategy** محددة بدل **AUTO** .

```
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String name;
}
```

مثال:-

```
Employee employee = new Employee();
employee.setName("Ahmed");

employeeRepository.save(employee);
```

ثم:-

Hibernate يقوم بتوليد **id** تلقائيًا.

Database

AUTO يمكن استخدامه مع قواعد بيانات مختلفة مثل:

- PostgreSQL
 - MySQL
 - Oracle
 - SQL Server
- لأن **Hibernate** هو الذي يختار آلية التوليد المناسبة.

GenerationType.IDENTITY

what is it?

هنا أنت تقول لـ: Hibernate

الـ **Database** نفسها مسؤولة عن توليد الـ ID باستخدام **Identity/Auto Increment**.

```
CREATE TABLE employee (  
  id BIGINT AUTO_INCREMENT,  
  name VARCHAR(255),  
  PRIMARY KEY (id)  
);
```

مثلاً في: MySQL

```
Employee employee = new Employee();  
employee.setName("Ahmed");  
repository.save(employee);
```

لو عملت:

أنت لا ترسل: **id = 1**

الـ **Database** هي التي تولده.

```
INSERT Employee  
↓  
Database  
↓  
Generate ID = 1
```

مثلاً:

```
id = 1  
name = Ahmed
```

ثم Employee يصبح:

```
id = 2  
name = Mohamed
```

والـ Employee التالي:

When to use it?

تستخدمها عندما:

استخدم **IDENTITY** عندما تكون الـ **Database** تعتمد على **auto-increment** **identity columns** / وتريد أن تكون الـ **Database** مسؤولة عن توليد الـ **IDs**.

وهي شائعة جداً مع:

MySQL .

MariaDB .

SQL Server .

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

مثلاً: MySQL

```
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

مثال:

```
Product product = new Product();
product.setName("Laptop");

productRepository.save(product);
```

عند:

Database يمكن أن تنتج:

id	name
1	Laptop

Database

مناسب جداً مع:

- MySQL
- SQL Server

نقطة مهمة جداً في الـ Interview

IDENTITY قد يؤثر على طريقة تنفيذ Hibernate للـ

INSERT لأن Hibernate يحتاج عادةً إلى تنفيذ الـ

INSERT للحصول على الـ generated ID.

وده يجعلها مختلفة عن SEQUENCE التي يمكن فيها

الحصول على ID من الـ sequence قبل تنفيذ الـ INSERT.

GenerationType.SEQUENCE

what is it?

دي مهمة جدًا خصوصًا مع **Oracle و PostgreSQL**

هنا الـ **Database** تستخدم **Sequence** لتوليد أرقام الـ **IDs**.

الـ **Sequence** عبارة عن **Database object** مسؤول عن إنتاج أرقام متتابعة.

```
CREATE SEQUENCE employee_seq  
START WITH 1  
INCREMENT BY 1;
```

في **Oracle** مثلاً:

ثم **Hibernate** يستخدم الـ **Sequence** للحصول على **ID**.

```
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.SEQUENCE)  
    private Long id;  
  
    private String name;  
}
```

مثال:

```
@Id  
@GeneratedValue(  
    strategy = GenerationType.SEQUENCE,  
    generator = "employee_seq"  
)  
@SequenceGenerator(  
    name = "employee_seq",  
    sequenceName = "employee_seq",  
    allocationSize = 1  
)  
private Long id;
```

أو تحدد **Sequence** معينة:

```
Hibernate
↓
employee_seq
↓
Get next ID
↓
INSERT Employee
```

هنا:

```
Sequence → 1
```

مثلاً:

```
INSERT:
id = 1
name = Ahmed
```

ثم:

```
Sequence → 2
```

```
INSERT:
id = 2
name = Mohamed
```

When to use it?

تستخدم **SEQUENCE** عندما تكون الـ **Database** تدعم **Sequences** وتريد الاستفادة منها.

خصوصاً:

• Oracle

• PostgreSQL

وهي اختيار ممتاز عندما يكون عندك عدد كبير من **Inserts** وتريد الاستفادة من آلية الـ **Sequence** والـ **ID allocation**.

Database

- Oracle

من أشهر الـ **Databases** التي تستخدم الـ **Sequences**.

- PostgreSQL

تدعم **Sequences** أيضاً.

```
CREATE SEQUENCE employee_seq  
START 1  
INCREMENT 1;
```

مثال:

نقطة مهمة جداً

SEQUENCE غالباً يمكن أن تكون أكثر مرونة من **IDENTITY** مع **Hibernate** في بعض سيناريوهات **batching** ، لأن **Hibernate** يستطيع الحصول على **IDs** من الـ **sequence** قبل تنفيذ الـ **INSERT**.

Sequence

↓

Get IDs

↓

1, 2, 3, 4

↓

Batch INSERT

مثلاً

وهذا يمكن أن يكون مفيداً في عمليات الإدخال الكثيرة.

GenerationType.TABLE

what is it?

دي مختلفة عن الثلاثة السابقين.

بدل ما تستخدم:

Identity .

Sequence .

Hibernate يستخدم جدولاً في الـ **Database** لتخزين وإدارة الـ **IDs**.

```
id_generator
```

```
-----  
sequence_name | next_val  
employee      | 10
```

يعني مثلاً **Hibernate** ينشئ جدولاً مثل:

```
Hibernate  
↓  
id_generator table  
↓  
Get next value  
↓  
Update value  
↓  
Use ID
```

ولما يحتاج **ID** جديد:

مثال:

```
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    private Long id;

    private String name;
}
```

يمكن أن يقوم **Hibernate** بإنشاء جدول خاص بتوليد الـ **IDs**

مثلاً بشكل تقريبي:

```
hibernate_sequences
-----
sequence_name | next_val
EMPLOYEE      | 1
```

وعند إضافة Employee:

```
next_val = 1
      ↓
Employee ID = 1
      ↓
next_val = 2
```

When to use it?

تاريخياً، كانت مفيدة عندما تريد طريقة مستقلة نسبياً عن **Database** **specific identity/sequence features.**

لكن في المشاريع الحديثة:

نادراً ما تكون الاختيار الأول.

لأنها تحتاج إلى عمليات إضافية على جدول الـ **generator** ، وقد تسبب

contention و **overhead** مقارنةً بالـ **native mechanisms** مثل

Identity أو **Sequence**.

Database

يمكن استخدامها مع **Databases** مختلفة لأنها تعتمد على **Table** بدل الاعتماد

على **Sequence** أو **Identity** خاص بالـ **Database**.

لكن في مشروع حديث، لو الـ **Database** تدعم **SEQUENCE** أو **IDENTITY**

بشكل جيد، غالباً ستفضل الـ **native strategy** المناسبة بدل **TABLE**.

الاستخدام	Database مناسبة	الآلية	مين بيولد ال ID؟	Strategy
عندما لا تحتاج التحكم في الطريقة	أي Database مدعومة	حسب ال Database	Hibernate يختار	<i>AUTO</i>
شائع وبسيط	MySQL, MariaDB, SQL Server	Identity / Auto Increment	Database	<i>IDENTITY</i>
ممتاز مع ال DB التي تدعم Sequences	Oracle, PostgreSQL	Sequence Object	Database	<i>SEQUENCE</i>
حالات خاصة، ونادرًا في المشاريع الحديثة	معظم ال Databases	Generator Table	Hibernate باستخدام Table	<i>TABLE</i>